

Redes de Datos

Práctica 2: Bloque Servicios

Francisco Javier Mercader Martínez | 26649110E | franciscojavier.mercader@edu.upct.es
Mauro Martínez Cazaux | 49273021Y | mauro.martinez@edu.upct.es

Tabla de contenidos

1. Servidor HTTP en Python para la página de entrada de la web	2
1.1. Servidor HTTP (webRD_python.py)	2
1.2. Salida del programa	14
1.3. Cuerpo del archivo index.html	16
1.4. Análisis de trazas con WireShark	17
2. Servicio Apache HTTP con el login y portal de desarrollo para los trabajadores de outlier	19

1. Servidor HTTP en Python para la página de entrada de la web

Para esta primera parte vamos a programar y desplegar un servicio HTTP como bien describe el enunciado, para ello explicaremos el proceso, es decir, el código programado y la salida de este.

1.1. Servidor HTTP (webRD_python.py)

Lo primero de todo sería importar las librerías necesarias para el programa, a continuación, se muestra cada una de ellas:

- **sys**: librería utilizada para leer los argumentos del terminal cuando queramos ejecutar el programa
- **urllib.parse**: librería utilizada para decodificar las URLs y también para leer los parámetros de la orden POST.
- **socket**: librería que nos permite trabajar con sockets (crea un servicio TCP que permite escuchar peticiones HTTP).
- **logging**: librería que nos permite generar un registro de salida para validar desde el terminal el estado del servicio TCP mientras está activo (salud, peticiones, timeouts, etc.).
- **datetime**: librería utilizada para el formateo de registros en los *headers* de las consultas HTTP en formato UTC.
- **os**: librería que nos permite levantar la interfaz visual de `index.html` junto con el propio servicio TCP al acceder al puerto 8080.
- **argparse**: librería que nos permite ajustar valores del servicio TCP en el shell a la hora de levantarlo sin necesidad de modificar su código fuente.
- **select**: librería utilizada para obtener las peticiones HTTP realizadas en la conexión TCP donde obtenemos los datos en el socket con timeout.

```
import argparse
import logging
import os
import select
import socket
import sys
from datetime import datetime, timezone
from urllib.parse import unquote_plus
```

Después definimos los parámetros necesarios para el tiempo de persistencia de HTTP y el nombre de la cookie.

XX representa los dos últimos dígitos del DNI de Francisco Javier Mercader Martínez y YY representa los dos últimos dígitos del DNI de Mauro Martínez Cazaux. Luego se le da un nombre a la cookie que utilizará el servidor para contar las visitas al archivo `index.html`, el nombre que le hemos dado es `cookie_counter_10_49`; y por último se define la suma del tiempo de espera del servidor HTTP como la suma de las cuatro cifras de XX e YY más 10 segundos, en nuestro caso son 24 segundos en total.

```
# =====
# CONFIGURACIÓN DE GRUPO
# =====
# XX y YY son las dos últimas cifras del DNI de cada miembro
XX = "10" # <-- Francisco Javier Mercader Martínez, DNI 26649110E
YY = "49" # <-- Mauro Martínez Cazaux, DNI 49273021Y

# Construcción automática de la cookie y del timeout según el enunciado.
# timeout = X + X + Y + Y + 10 (suma de cada una de las cuatro cifras)
COOKIE_NAME = f"cookie_counter_{XX}_{YY}"
TIMEOUT_CONNECTION = sum(int(c) for c in (XX + YY) if c.isdigit()) + 10
```

A continuación se declaran las constantes generales del servidor. `BUFSIZE` indica el tamaño máximo de cada bloque de datos que se recibe o se envía por el socket. En este caso se usa un valor de 8192 bytes, suficiente para trabajar con peticiones HTTP sencillas y para ir enviando los ficheros por partes.

`MAX_ACCESOS` establece el número máximo de accesos permitidos a la página `index.html` antes de bloquear temporalmente al cliente mediante una respuesta 403 `Forbidden`. `COOKIE_MAX_AGE` indica la duración de la cookie de control de accesos, que en este caso es de 120 segundos, es decir, 2 minutos.

También se crea el diccionario `filetypes`, donde se relacionan las extensiones de los ficheros estáticos con su tipo MIME. Esta información es necesaria para rellenar la cabecera `Content-Type`, de forma que el navegador sepa si el recurso recibido debe interpretarse como HTML, CSS, JavaScript o una imagen.

```
# =====
# CONSTANTES
# =====
BUFSIZE = 8192 # Tamaño máximo de buffer
MAX_ACCESOS = 5 # Accesos máximos antes de bloquear
COOKIE_MAX_AGE = 120 # Caducidad de la cookie (2 minutos)

# Extensiones admitidas (ext, MIME)
filetypes = {
    "gif": "image/gif", "jpg": "image/jpg",
    "jpeg": "image/jpeg", "png": "image/png",
    "htm": "text/htm", "html": "text/html",
    "css": "text/css", "js": "text/js",
}
```

Después se configura el sistema de registros del servidor. Esta parte permite ver en el terminal qué está ocurriendo mientras el servicio está activo: cuándo se acepta una conexión, qué línea de petición llega, qué valor tiene la cookie, si se alcanza un timeout o si se produce algún error.

El formato elegido muestra la fecha, los milisegundos, el nivel del mensaje y el identificador del proceso. Este último dato es útil porque el servidor atiende clientes con procesos hijo creados mediante `os.fork()`, por lo que pueden aparecer varios procesos escribiendo mensajes al mismo tiempo.

```
# =====
# LOGGING
# =====
logging.basicConfig(
    level=logging.INFO,
    format="[%(asctime)s.%(msecs)03d] [%(levelname)-7s] "
    "[pid=%(process)d] %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)
logger = logging.getLogger()
```

Las primeras funciones auxiliares son las funciones básicas de comunicación con el socket. La función `enviar_mensaje()` utiliza `sendall()` para asegurarse de que todos los bytes de la respuesta se mandan al cliente. Esto es importante porque una llamada normal a `send()` no garantiza que se envíe todo el contenido en una única operación.

La función `recibir_mensaje()` lee datos del cliente usando el tamaño máximo definido en `BUFSIZE`, mientras que `cerrar_conexion()` cierra el socket de forma controlada. En esta última función se usa un bloque `try-except` para evitar que un error al cerrar una conexión ya cerrada detenga el servidor.

```
# =====
# FUNCIONES BASICAS DE SOCKET
# =====
def enviar_mensaje(cs, data):
    """
    Envía datos a través del socket cs. Devuelve número de bytes enviados.
    """
    cs.sendall(data)
    return len(data)

def recibir_mensaje(cs):
    """Recibe datos a través del socket cs. Devuelve un objeto bytes."""
    return cs.recv(BUFSIZE)

def cerrar_conexion(cs):
    """Cierra una conexión activa."""
    try:
        cs.close()
    except Exception:
```

`pass`

La siguiente función auxiliar es `http_date()`. Su objetivo es generar la fecha actual en el formato utilizado por HTTP en la cabecera `Date`. Para ello se toma la hora actual en UTC y se formatea con el patrón habitual de las respuestas HTTP, terminando en GMT.

```
# =====
# UTILIDAD: Fecha en formato HTTP (RFC 7231)
# =====
def http_date():
    return datetime.now(timezone.utc).strftime("%a, %d %b %Y %H: %M: %S GMT")
```

Después aparece la función `enviar_error()`, que centraliza la construcción de respuestas HTTP de error. Esta función recibe el socket del cliente, el código de estado, el texto asociado al código, un mensaje explicativo y un parámetro `keep_alive` que indica si la conexión debe mantenerse abierta.

Primero se construye un pequeño documento HTML con el código de error, el motivo y el mensaje que se quiere mostrar al usuario. Ese cuerpo se codifica en UTF-8 para poder enviarlo por el socket.

```
def enviar_error(cs, code, reason, message, keep_alive=False):
    """Envía una respuesta HTTP de error con una pagina HTML informativa."""
    body = (
        f'<html><head><meta charset="UTF-8"><title>{code} {reason}</title></head>'
        f'<body><h1>{code} {reason}</h1><p>{message}</p>'
        f'<hr><i>webRD_python</i></body></html>'
    ).encode("utf-8")
```

A continuación se decide qué cabecera `Connection` se va a utilizar. Si el cliente puede mantener la conexión abierta, se envía `Connection: keep-alive` junto con la cabecera `Keep-Alive`, donde se indica el timeout calculado anteriormente. Si no, se informa de que la conexión debe cerrarse.

```
if keep_alive:
    connection_h = (
        "Connection: keep-alive\r\n"
        f"Keep-Alive: timeout={TIMEOUT_CONNECTION}\r\n"
    )
else:
    connection_h = "Connection: close\r\n"
```

Por último, se construyen las cabeceras completas de la respuesta HTTP. Se incluye la versión y el código de estado, la fecha, el nombre del servidor, la longitud del cuerpo y el tipo de contenido. Después se envían cabeceras y cuerpo en una sola llamada a `enviar_mensaje()`.

```
headers = (
    f"HTTP/1.1 {code} {reason}\r\n"
    f>Date: {http_date()}\r\n"
    "Server: webRD_python\r\n"
    f"{connection_h}"
    f"Content-Length: {len(body)}\r\n"
    "Content-Type: text/html; charset=utf-8\r\n"
    "\r\n"
).encode("utf-8")

enviar_mensaje(cs, headers + body)
```

La función `process_cookies()` se encarga de gestionar la cookie de control de accesos. Recibe la lista de cabeceras de la petición HTTP y busca una cabecera que empiece por `Cookie:`. Como una misma cabecera puede contener varias cookies separadas por punto y coma, se recorre cada una hasta encontrar la que coincide con `COOKIE_NAME`.

Si no existe la cookie, significa que es el primer acceso del cliente y se devuelve el valor 1. Si la cookie existe y su valor es menor que `MAX_ACCESOS`, se incrementa en uno. Si ya ha llegado al máximo, se devuelve `MAX_ACCESOS` para que el servidor pueda bloquear el acceso a la página principal.

```
# =====
# COOKIES
# =====
def process_cookies(headers, cs):
    """
    Procesa la cabecera Cookie en busca de la cookie de control de accesos.

    Reglas:
    * Si no se encuentra cookie_counter_XXYY -> devuelve 1 (primer acceso).
    * Si esta y vale MAX_ACCESOS -> devuelve MAX_ACCESOS.
    * Si esta y 1 <= valor < MAX_ACCESOS -> devuelve valor + 1.
    """
    for h in headers:
        if h.lower().startswith("cookie:"):
            valor_completo = h.split(":", 1)[1].strip()
            # Pueden coexistir varias cookies separadas por ";"
            for cookie in valor_completo.split(";"):
                cookie = cookie.strip()
                if cookie.startswith(COOKIE_NAME + "="):
                    try:
                        n = int(cookie.split("=", 1)[1])
                    except ValueError:
                        return 1
                    if n >= MAX_ACCESOS:
                        return MAX_ACCESOS
                    return n + 1
    return 1
```

La siguiente función es `procesar_post_form()`, encargada de tratar el formulario enviado mediante el método POST. El formulario usa el formato `application/x-www-form-urlencoded`, por lo que el cuerpo de la petición llega como una cadena de pares `clave=valor` separados por `&`.

El servidor busca el campo `email`, decodifica su valor con `unquote_plus()` y comprueba si el correo termina en el dominio `@edu.upct.es`. De este modo se valida si el correo introducido pertenece a la Universidad Politécnica de Cartagena.

```
# =====
# RESPUESTA A POST: validacion del formulario
# =====
def procesar_post_form(cs, body_text, keep_alive):
    """
    Procesa el formulario enviado via POST con application/x-www-form-urlencoded.
    Espera un campo 'email'. Valida si el dominio es @edu.upct.es.
    """
    email = ""
    for par in body_text.split("&"):
        if "=" in par:
            k, v = par.split("=", 1)
            if k.lower() == "email":
                email = unquote_plus(v)
                break

    correcto = email.lower().endswith("@edu.upct.es")
```

Si el correo es correcto se genera una página HTML indicando que pertenece al dominio permitido. Si no es correcto, se genera otra página indicando que el correo no pertenece a dicho dominio. En el caso de que no se haya recibido ningún correo, se muestra el texto (vacío).

```
if correcto:
    body = (
        '<html><head><meta charset="UTF-8">'
        '<title>Correo correcto</title></head>'
    )
```

```

        "<body><h1>Correo correcto</h1>"
        f"<p>El correo <b>{email}</b> pertenece al dominio "
        "<code>@edu.upct.es</code>.</p>"
        '<p><a href="/">Volver al inicio</a></p>'
        "</body></html>"
    ).encode("utf-8")
else:
    mostrado = email if email else "(vacio)"
    body = (
        '<html><head><meta charset="UTF-8">'
        "<title>Correo incorrecto</title></head>"
        "<body><h1>Correo incorrecto</h1>"
        f"<p>El correo <b>{mostrado}</b> NO pertenece al dominio "
        "<code>@edu.upct.es</code>.</p>"
        '<p><a href="/">Volver al inicio</a></p>'
        "</body></html>"
    ).encode("utf-8")

```

Al igual que ocurría en las respuestas de error, la función prepara las cabeceras HTTP según se mantenga o no la conexión persistente. En este caso el código de estado siempre es 200 OK, porque el formulario se ha procesado correctamente aunque el correo no sea válido.

```

if keep_alive:
    connection_h = (
        "Connection: keep-alive\r\n"
        f"Keep-Alive: timeout={TIMEOUT_CONNECTION}\r\n"
    )
else:
    connection_h = "Connection: close\r\n"

headers = (
    "HTTP/1.1 200 OK\r\n"
    f>Date: {http_date()}\r\n"
    "Server: webRD_python\r\n"
    f"{connection_h}"
    f"Content-Length: {len(body)}\r\n"
    "Content-Type: text/html; charset=utf-8\r\n"
    "\r\n"
).encode("utf-8")

enviar_mensaje(cs, headers + body)

```

La parte principal del servidor se concentra en la función `process_web_request()`. Esta función recibe el socket de cliente `cs` y el directorio base `files`, desde el que se servirán los ficheros estáticos.

Su funcionamiento se basa en un bucle que permite procesar una o varias peticiones HTTP dentro de la misma conexión TCP. Esto implementa las conexiones persistentes de HTTP/1.1. El bucle termina cuando se alcanza el timeout, cuando el cliente cierra la conexión, cuando se recibe `Connection: close` o cuando aparece un error grave.

```

# =====
# PROCESAMIENTO PRINCIPAL DE LA PETICION
# =====
def process_web_request(cs, files):
    """
    Procesa una o varias peticiones HTTP en una conexión TCP.

    Bucle Keep-Alive con select() y timeout. Sale del bucle cuando:
    * Se alcanza TIMEOUT_CONNECTION sin recibir datos
    * El cliente cierra la conexión
    * Se recibe Connection: close
    * Ocurre un error grave
    """

```

```
while True:
```

Lo primero que se hace dentro del bucle es esperar datos usando `select.select()`. La llamada recibe el socket del cliente y el tiempo máximo de espera. Si durante ese tiempo no llega ninguna petición, el servidor escribe un mensaje en el log y cierra la conexión. Esta solución evita que un cliente mantenga una conexión abierta indefinidamente sin enviar datos.

```
# -----
# 1) Esperar datos en el socket con timeout (Keep-Alive)
# -----
rlist, _, _ = select.select([cs], [], [], TIMEOUT_CONNECTION)
if not rlist:
    logger.info("Timeout de conexion alcanzado, cerrando...")
    break
```

Después se leen los datos recibidos mediante `recibir_mensaje()`. Si no se recibe nada, significa que el cliente ha cerrado la conexión. En caso contrario, los bytes recibidos se decodifican como UTF-8. Si ocurre algún problema durante la decodificación, el servidor responde con un error 400 Bad Request.

```
# -----
# 2) Leer la petición
# -----
raw = recibir_mensaje(cs)
if not raw:
    logger.info("Cliente cerro la conexion (recv vacio).")
    break

try:
    req_text = raw.decode("utf-8", errors="replace")
except Exception as e:
    logger.error(f"Error decodificando petición: {e}")
    enviar_error(cs, 400, "Bad Request", "Mensaje no decodificable")
    break
```

Una petición HTTP está formada por una zona de cabeceras y, opcionalmente, un cuerpo. Ambas partes se separan mediante la secuencia `\r\n\r\n`. Si dicha secuencia aparece en la petición, se separan cabeceras y cuerpo. Si no aparece, se considera que la petición solo contiene cabeceras.

```
# -----
# 3) Separar cabeceras y body
# -----
if "\r\n\r\n" in req_text:
    header_text, body_text = req_text.split("\r\n\r\n", 1)
else:
    header_text = req_text
    body_text = ""

lines = header_text.split("\r\n")
if not lines or not lines[0].strip():
    enviar_error(cs, 400, "Bad Request", "Petición vacía")
    break

request_line = lines[0]
logger.info(f"Request line: {request_line}")
```

La primera línea de la petición es la línea de solicitud. Debe tener tres partes: método, ruta y versión HTTP. Por ejemplo: `GET /index.html HTTP/1.1`. Si no tiene exactamente tres partes, el servidor devuelve 400 Bad Request.

Además, este servidor solo acepta HTTP/1.1. Si el cliente usa otra versión, se responde con 505 HTTP Version Not Supported. Por último, solo se aceptan los métodos GET y POST; cualquier otro método produce un error 405 Method Not Allowed.

```

# -----
# 4) Parsear línea de solicitud: METHOD PATH VERSION
# -----
partes = request_line.split()
if len(partes) != 3:
    enviar_error(
        cs, 400, "Bad Request", "Línea de solicitud mal formada"
    )
    break
method, path, version = partes

if version != "HTTP/1.1":
    enviar_error(
        cs,
        505,
        "HTTP Version Not Supported",
        f"Solo se soporta HTTP/1.1 (recibido {version})",
    )
    break

if method not in ("GET", "POST"):
    enviar_error(
        cs, 405, "Method Not Allowed", f"Metodo {method} no permitido"
    )
    break

```

Después se revisa la cabecera `Connection`. En HTTP/1.1 las conexiones persistentes están activadas por defecto, por eso `keep_alive` comienza con el valor `True`. Si el cliente envía `Connection: close`, la conexión se cerrará al terminar de responder. Si envía `Connection: keep-alive`, se mantiene abierta para aceptar más peticiones dentro del timeout configurado.

```

# -----
# 5) Detectar Connection (keep-alive por defecto en HTTP/1.1)
# -----
keep_alive = True # Por defecto en HTTP/1.1
for h in lines[1:]:
    if h.lower().startswith("connection:"):
        valor = h.split(":", 1)[1].strip().lower()
        if "close" in valor:
            keep_alive = False
        elif "keep-alive" in valor:
            keep_alive = True

# Logging de cabeceras (modo verbose)
for h in lines[1:]:
    if h.strip():
        logger.debug(f"Header: {h}")

```

Si el método recibido es `POST`, el servidor busca la cabecera `Content-Length`, ya que indica cuántos bytes ocupa el cuerpo de la petición. Esto permite comprobar si el cuerpo se ha recibido completo. Si todavía faltan datos, el servidor sigue leyendo del socket hasta completar el tamaño esperado o hasta que el cliente cierre la conexión.

Una vez reconstruido el cuerpo completo, se llama a `procesar_post_form()` para validar el correo electrónico. Si la conexión no es persistente, se sale del bucle; si lo es, el servidor continúa esperando nuevas peticiones.

```

# -----
# 6) POST: procesar formulario
# -----
if method == "POST":
    content_length = 0
    for h in lines[1:]:
        if h.lower().startswith("content-length"):

```



```

        try:
            content_length = int(h.split(":", 1)[1].strip())
        except ValueError:
            content_length = 0
        break

# Asegurar que tenemos todo el body
while len(body_text.encode("utf-8")) < content_length:
    more = recibir_mensaje(cs)
    if not more:
        break
    body_text += more.decode("utf-8", errors="replace")

logger.info(f"POST body: {body_text}")
procesar_post_form(cs, body_text, keep_alive)

if not keep_alive:
    break
continue

```

Si el método recibido es GET, el servidor debe localizar y enviar un fichero estático. Primero elimina la parte de parámetros de la URL, es decir, todo lo que aparece a partir de ?. Después transforma la ruta / en `index.html`, para que al entrar en la raíz del servidor se cargue la página principal.

También se añade una comprobación básica de seguridad para evitar accesos fuera del directorio permitido. Si la ruta contiene `..`, empieza por una barra invertida o contiene `:`, se rechaza con `403 Forbidden`. Así se evita, por ejemplo, que un cliente intente leer ficheros del sistema usando rutas relativas maliciosas.

```

# -----
# 7) GET: servir fichero estatico
# -----
# Eliminar query string
if "?" in path:
    path = path.split("?", 1)[0]

# / -> index.html
if path == "/":
    resource = "index.html"
else:
    resource = path.lstrip("/")

# Seguridad minima: rechazar path traversal
if ".." in resource or resource.startswith("\\") or ":" in resource:
    enviar_error(
        cs, 403, "Forbidden", "Acceso al recurso denegado", keep_alive
    )
    if not keep_alive:
        break
    continue

```

Con la ruta ya validada, se construye el path final combinando el directorio base `files` con el recurso solicitado. Si el fichero no existe, se responde con `404 Not Found`. En caso de que la conexión sea persistente, el servidor no termina inmediatamente, sino que queda preparado para recibir otra petición del mismo cliente.

```

filepath = os.path.join(files, resource)

if not os.path.isfile(filepath):
    enviar_error(
        cs,
        404,
        "Not Found",
        f"El recurso solicitado no existe: /{resource}",
    )

```

```

        keep_alive,
    )
    if not keep_alive:
        break
    continue

```

La gestión de cookies solo se aplica al fichero `index.html`. Cuando el cliente solicita esta página, se llama a `process_cookies()` para obtener el nuevo valor del contador. Este valor se registra por consola para poder comprobar el funcionamiento del servidor.

Si el contador llega a `MAX_ACCESOS`, el servidor no entrega la página principal. En su lugar, genera una respuesta 403 Forbidden con una página HTML informativa. Además, vuelve a enviar la cookie con el valor máximo y con `Max-Age=120`, de forma que el bloqueo se mantenga durante dos minutos.

```

# -----
# 8) Cookies: solo se incrementan al pedir index.html
# -----
cookie_header = b""
if resource == "index.html":
    n = process_cookies(lines[1:], cs)
    logger.info(f"{COOKIE_NAME} = {n}")

    if n >= MAX_ACCESOS:
        # Bloqueo: 403 Forbidden con pagina informativa
        body = (
            '<html><head><meta charset="UTF-8">'
            '<title>403 Forbidden</title></head>'
            '<body><h1>403 Forbidden</h1>'
            f'<p>Has alcanzado el numero maximo de accesos permitidos '
            f'(<b>{MAX_ACCESOS}</b>) a esta pagina.</p>'
            f'<p>Espera 2 minutos para que expire la cookie.</p>'
            '<hr><i>webRD_python</i></body></html>'
        ).encode("utf-8")

```

Para la respuesta de bloqueo también se respetan las conexiones persistentes. Se preparan las cabeceras HTTP, se añade `Set-Cookie` con el valor máximo y se envía todo al cliente. Después, si la conexión sigue siendo persistente, el servidor vuelve a esperar nuevas peticiones; en caso contrario, se cierra.

```

if keep_alive:
    connection_h = (
        "Connection: keep-alive\r\n"
        f"Keep-Alive: timeout={TIMEOUT_CONNECTION}\r\n"
    )
else:
    connection_h = "Connection: close\r\n"

headers = (
    "HTTP/1.1 403 Forbidden\r\n"
    f"Date: {http_date()}\r\n"
    "Server: webRD_python\r\n"
    f"{connection_h}"
    f"Set-Cookie: {COOKIE_NAME}={MAX_ACCESOS}; "
    f"Max-Age={COOKIE_MAX_AGE}\r\n"
    f"Content-Length: {len(body)}\r\n"
    "Content-Type: text/html; charset=utf-8\r\n"
    "\r\n"
).encode("utf-8")
enviar_mensaje(cs, headers + body)

if not keep_alive:
    break
continue

```

Si todavía no se ha alcanzado el límite de accesos, se prepara la cabecera **Set-Cookie** con el nuevo contador. Esta cabecera se añadirá a la respuesta 200 OK, por lo que el navegador almacenará el nuevo valor y lo enviará en peticiones posteriores.

```
cookie_header = (
    f"Set-Cookie: {COOKIE_NAME}={n}; Max-Age={COOKIE_MAX_AGE}\r\n"
).encode("utf-8")
```

Una vez superadas todas las comprobaciones, el servidor construye la respuesta correcta. Primero obtiene la extensión del fichero solicitado y busca su tipo MIME en el diccionario `filetypes`. Si la extensión no está registrada, se usa `application/octet-stream`, que indica contenido binario genérico.

También se calcula el tamaño del fichero con `os.path.getsize()`, ya que ese valor se debe enviar en la cabecera **Content-Length**. Si el contenido es de tipo texto, se añade `charset=utf-8` al **Content-Type**.

```
# -----
# 9) Construir y enviar respuesta 200 OK
# -----
ext = filepath.rsplit(".", 1)[-1].lower()
content_type = filetypes.get(ext, "application/octet-stream")
size = os.path.getsize(filepath)

if keep_alive:
    connection_h = (
        "Connection: keep-alive\r\n"
        f"Keep-Alive: timeout={TIMEOUT_CONNECTION}\r\n"
    ).encode("utf-8")
else:
    connection_h = b"Connection: close\r\n"

# Charset solo para texto
ct_header = content_type
if content_type.startswith("text/"):
    ct_header += "; charset=utf-8"
```

Después se montan las cabeceras de la respuesta 200 OK. En ellas se incluye la fecha, el servidor, la política de conexión, la longitud del fichero, el tipo de contenido y, si corresponde, la cabecera de cookie preparada anteriormente.

```
headers = (
    (
        "HTTP/1.1 200 OK\r\n"
        f"Date: {http_date()}\r\n"
        "Server: webRD_python\r\n"
    ).encode("utf-8")
    + connection_h
    + (
        f"Content-Length: {size}\r\nContent-Type: {ct_header}\r\n"
    ).encode("utf-8")
    + cookie_header
    + b"\r\n"
)

enviar_mensaje(cs, headers)
```

Finalmente se abre el fichero en modo binario y se envía por bloques de `BUFSIZE` bytes. Esto permite servir tanto ficheros de texto como imágenes sin tener que cargar todo el contenido en memoria de una sola vez.

```
# Enviar cuerpo en bloques de BUFSIZE bytes
with open(filepath, "rb") as f:
    while True:
        chunk = f.read(BUFSIZE)
        if not chunk:
```

```

        break
    enviar_mensaje(cs, chunk)

```

Al terminar de enviar la respuesta se comprueba si la conexión debe mantenerse abierta. Si el cliente había pedido `Connection: close`, el bucle se rompe y se cierra el socket. Si no, el servidor vuelve al principio del bucle para esperar otra petición sobre la misma conexión TCP.

```

# -----
# 10) Salir si no es keep-alive
# -----
if not keep_alive:
    break

cerrar_conexion(cs)

```

La última parte del programa es la función `main()`, que arranca el servidor TCP y acepta conexiones de clientes. Primero se crea un parser de argumentos con `argparse`. El servidor necesita recibir el puerto con `-p` o `--port`, la dirección IP con `-ip` o `--host`, y opcionalmente el directorio base de ficheros con `-f` o `--files`. También se incluye la opción `--verbose` o `-v` para mostrar mensajes de depuración.

```

# =====
# MAIN: servidor concurrente con fork
# =====
def main():
    """Servidor web TCP concurrente."""
    try:
        parser = argparse.ArgumentParser()
        parser.add_argument(
            "-p", "--port", type=int, required=True, help="Puerto del servidor"
        )
        parser.add_argument(
            "-ip",
            "--host",
            required=True,
            help="Direccion IP del servidor o localhost",
        )
        parser.add_argument(
            "-f",
            "--files",
            help="Directorio base desde donde se sirven los ficheros",
        )
        parser.add_argument(
            "--verbose",
            "-v",
            action="store_true",
            help="Imprimir mensajes de depuracion",
        )
        args = parser.parse_args()

```

Si se activa el modo `verbose`, el nivel de logging pasa a `DEBUG`, por lo que también se muestran las cabeceras recibidas. Después se imprimen por consola los datos principales de configuración: dirección, puerto, directorio servido, nombre de la cookie y timeout de conexión.

```

if args.verbose:
    logger.setLevel(logging.DEBUG)

logger.info(
    f"Enabling server in address {args.host} and port {args.port}."
)
logger.info(f"Serving files from {args.files}")
logger.info(f"COOKIE_NAME = {COOKIE_NAME}")

```

```
logger.info(f"TIMEOUT_CONNECTION = {TIMEOUT_CONNECTION}s")
```

A continuación se crea el socket TCP. Se usa `socket.AF_INET` para trabajar con IPv4 y `socket.SOCK_STREAM` para usar TCP. La opción `SO_REUSEADDR` permite reutilizar la dirección aunque el socket anterior haya quedado temporalmente en estado de espera tras cerrar el servidor. Después se enlaza el socket con la IP y el puerto indicados y se pone en modo escucha con `listen()`.

```
# -----
# Crear socket TCP, permitir reuso de direccion, bind y listen
# -----
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
sock.bind((args.host, args.port))
sock.listen()
```

El servidor entra entonces en un bucle infinito en el que acepta conexiones con `accept()`. Cada vez que llega un cliente, se obtiene un nuevo socket conectado `cs` y la dirección del cliente `addr`.

Antes de crear un nuevo proceso hijo, el servidor intenta recoger procesos hijos anteriores que ya hayan terminado. Para ello usa `os.waitpid(-1, os.WNOHANG)`, que permite limpiar procesos finalizados sin bloquear el proceso padre.

```
# -----
# Bucle principal: aceptar conexiones y bifurcar el proceso
# -----
while True:
    cs, addr = sock.accept()
    logger.info(f"Accepted connection from {addr}")

    # Recolectar zombies de hijos previos sin bloquear
    try:
        while True:
            wpid, _ = os.waitpid(-1, os.WNOHANG)
            if wpid == 0:
                break
    except (ChildProcessError, OSError):
        pass
```

Para atender a varios clientes de forma concurrente se utiliza `os.fork()`. Esta llamada crea un proceso hijo. Si el sistema no soporta `fork()`, por ejemplo en Windows, el programa cae a un modo iterativo donde atiende la conexión sin crear un proceso nuevo.

```
# fork() solo existe en sistemas POSIX (Linux contenedor Docker).
# Si se ejecuta en Windows, se cae a modo iterativo.
try:
    pid = os.fork()
except (AttributeError, OSError):
    logger.warning("os.fork() no disponible, modo iterativo.")
    try:
        process_web_request(cs, args.files)
    except Exception as e:
        logger.exception(f"Error procesando petición: {e}")
    finally:
        cerrar_conexion(cs)
    continue
```

Cuando `fork()` devuelve 0, significa que el código se está ejecutando en el proceso hijo. Este proceso cierra el socket de escucha heredado del padre, atiende la petición mediante `process_web_request()` y termina con `sys.exit(0)`. Así cada hijo se dedica únicamente al cliente que le ha tocado.

Cuando `fork()` devuelve un identificador distinto de cero, el código está en el proceso padre. El padre no atiende directamente al cliente, por lo que cierra su copia del socket conectado y vuelve al bucle para aceptar nuevas conexiones.

```

if pid == 0:
    # PROCESO HIJO: cierra el socket de escucha del padre,
    # atiende la petición y termina.
    sock.close()
    try:
        process_web_request(cs, args.files)
    except Exception as e:
        logger.exception(f"Error procesando petición: {e}")
    finally:
        cerrar_conexion(cs)
    sys.exit(0)
else:
    # PROCESO PADRE: cierra el socket conectado y vuelve a esperar
    cerrar_conexion(cs)

```

Por último, `main()` captura dos situaciones generales. Si el usuario detiene el servidor con **Ctrl+C**, se muestra un mensaje indicando que el servicio se ha parado. Si aparece cualquier otro error no controlado, se registra con `logger.exception()`, que además de mostrar el mensaje incluye la traza del fallo.

La última condición hace que `main()` solo se ejecute cuando el archivo se lanza directamente como programa principal.

```

except KeyboardInterrupt:
    logger.info("Servidor detenido por el usuario.")
except Exception as e:
    logger.exception(f"Error en main: {e}")

if __name__ == "__main__":
    main()

```

Para ejecutar este servidor desde terminal, se indica el puerto, la IP y el directorio donde se encuentran los ficheros de la web. En nuestro caso, una ejecución típica sería:

```
python3 webRD_python.py -p 8080 -ip 0.0.0.0 -f files
```

Con esta orden, el servidor queda escuchando peticiones HTTP en el puerto 8080 y sirve los recursos del directorio `files`. Al acceder desde un navegador a la ruta `/`, se entrega `index.html`; al enviar el formulario de correo, se procesa una petición POST; y al refrescar varias veces la página principal, la cookie `cookie_counter_10_49` va aumentando hasta bloquear temporalmente el acceso al llegar al máximo configurado.

1.2. Salida del programa

Aquí vamos a mostrar los resultados en la terminal y en la página HTML.

Como se puede comprobar si ponemos una mala sintaxis para iniciar el servidor nos devuelve el mensaje “Usage: `webRD_python.py [-h] -p PORT -ip HOST [-f FILES] [--verbose]`”.

Una vez puesta la sintaxis correctamente, se muestra por pantalla que el servidor se ha iniciado en la URL <http://localhost:8080>, se ha usado el puerto 8080 ya que está libre.

```

> python3 webRD_python.py
usage: webRD_python.py [-h] -p PORT -ip HOST [-f FILES] [--verbose]
webRD_python.py: error: the following arguments are required: -p/--port, -ip/--host

> python3 webRD_python.py -p 8080 -ip 0.0.0.0 -f files
[2026-05-01 12:01:04.047] [INFO] [pid=238984] Enabling server in address 0.0.0.0 and port 8080.
[2026-05-01 12:01:04.047] [INFO] [pid=238984] Serving files from files
[2026-05-01 12:01:04.047] [INFO] [pid=238984] COOKIE_NAME = cookie_counter_10_49
[2026-05-01 12:01:04.047] [INFO] [pid=238984] TIMEOUT_CONNECTION = 24s

```

A continuación mostraremos una pequeña parte de nuestra página, para ellos buscamos en el navegador la URL que nos proporciona la terminal.

puerto 8080 - HTTP/1.1

SERVIDOR ACTIVO

Validador correo UPCT

Servidor HTTP/1.1 en Python con soporte keep-alive, cookies, formularios y servicio de ficheros estáticos.

PROTOCOLO
HTTP/1.1

PUERTO
8080

MAX ACCESOS
5 / cookie

KEEP-ALIVE
200 s

// RECURSO ESTÁTICO - IMAGEN.PNG



Vamos a mostrar ahora que funcionan correctamente los mensajes de error, así como las cookies.

Si se pone un correo que contiene la destinación @edu.upct.es entonces se muestra en la pantalla que es correcto y ha sido verificado. En caso contrario, muestra que es incorrecto y que se vuelva a intentar.

// ENDPOINT POST - /ACCION_FORM.HTML

Validación de correo UPCT

Comprueba si el correo pertenece al dominio @edu.upct.es

CORREO ELECTRÓNICO

fcojavier.mercader04@gmail.com

Enviar →

// ENDPOINT POST - /ACCION_FORM.HTML

Validación de correo UPCT

Comprueba si el correo pertenece al dominio @edu.upct.es

CORREO ELECTRÓNICO

franciscojavier.mercader@edu.upct.es

Enviar →

Correo incorrecto

El correo **fcojavier.mercader04@gmail.com** NO pertenece al dominio @edu.upct.es.

[Volver al inicio](#)

Correo correcto

El correo **franciscojavier.mercader@edu.upct.es** pertenece al dominio @edu.upct.es.

[Volver al inicio](#)

Una vez se hayan realizado peticiones habiendo puesto 5 emails, si decidimos volver nos saldrá el error 403, donde devolverá que hay demasiadas peticiones realizadas y que se debe esperar 2 minutos para volver a intentarlo.

403 Forbidden

Has alcanzado el numero maximo de accesos permitidos (5) a esta pagina.

Espera 2 minutos para que expire la cookie.

webRD_python

1.3. Cuerpo del archivo index.html

Dentro del archivo `index.html` se encuentra el cuerpo de nuestra página web donde se puede observar el título de esta (“Validador UPCT”), todas las *stats* correspondientes al servicio TCP, la imagen adjuntada como recursos estático, el campo de entrada del email con las características descritas en los apartados anteriores y un pequeño apartado a pie de página que muestra el nombre del archivo del servidor (`webRD_python`), la universidad (UPCT), la asignatura y los autores.

```
<body>
  <header>
    <div class="logo-dot"></div>
    <div class="logo-text"><span>puerto 8080</span> &mdash; HTTP/1.1</div>
  </header>

  <main>
    <!-- Hero -->
    <section class="hero">
      <div class="tag">servidor activo</div>
      <h1>Validador correo <em>UPCT</em></h1>
      <p class="subtitle">
        Servidor HTTP/1.1 en Python con soporte keep-alive, cookies, formularios
        y servicio de ficheros estáticos.
      </p>
    </section>

    <!-- Stats -->
    <div class="stats-grid">
      <div class="stat">
        <div class="stat-label">Protocolo</div>
        <div class="stat-val">HTTP/1.1</div>
      </div>
      <div class="stat">
        <div class="stat-label">Puerto</div>
        <div class="stat-val">8080</div>
      </div>
      <div class="stat">
        <div class="stat-label">Max accesos</div>
        <div class="stat-val">5 / cookie</div>
      </div>
      <div class="stat">
        <div class="stat-label">Keep-Alive</div>
        <div class="stat-val">200 s</div>
      </div>
    </div>
  </div>
```



```

<!-- Imagen -->
<section class="section">
  <div class="section-label">recurso estático &mdash; imagen.png</div>
  <div class="img-card">
    
  </div>
</section>

<!-- Formulario -->
<section class="section">
  <div class="section-label">endpoint POST &mdash; /accion_form.html</div>
  <div class="form-card">
    <h2>Validación de correo UPCT</h2>
    <p class="form-desc">
      Comprueba si el correo pertenece al dominio @edu.upct.es
    </p>
    <form action="/accion_form.html" method="POST">
      <div class="field">
        <label for="email">Correo electrónico</label>
        <div class="input-row">
          <input
            type="text"
            id="email"
            name="email"
            placeholder="usuario@edu.upct.es"
          />
          <button type="submit">Enviar &mdash;</button>
        </div>
      </div>
    </form>
  </div>
</section>

<!-- Cookie info -->
<div class="cookie-info">
  <span class="cookie-icon"> </span>
  <div class="cookie-text">
    <strong>Control de acceso por cookie</strong><br />
    Esta página registra tus visitas mediante la cookie
    <code style="color: #79c0ff">cookie_counter_1234</code>. Después de
    <strong>5 accesos</strong> se bloqueará el acceso durante 2 minutos.
  </div>
</div>
</main>

<footer>
  <span>webRD_python &copy; 2026 &mdash; UPCT</span>
  <span>Redes de Datos</span>
  <span> Fco. Javier Mercader & Mauro Martínez </span>
</footer>
</body>

```

1.4. Análisis de trazas con WireShark

En esta parte, se van a mostrar las trazas con WireShark y posteriormente se va a dar una explicación sobre estas.

Vamos a analizar primero las trazas del método **GET** donde se puede observar que hay varias debido a que el método GET se encarga en HTTP de solicitar o recuperar un recurso del servidor. Como hemos dicho antes se puede ver que utiliza la versión HTTP/1.1.

En cuanto a las peticiones que hace es, por ejemplo, la primera trama es la petición que se le envía al navegador para cargar la página principal del servidor `localhost:8080`. La segunda trama es la petición de la imagen puesta.

No.	Time	Source	Destination	Protocol	Length	Info
276	15.848328800	:::1	:::1	HTTP	1664	GET / HTTP/1.1
284	15.861523393	:::1	:::1	HTTP	1587	GET /imagen.png HTTP/1.1
411	52.205176411	:::1	:::1	HTTP	1688	GET / HTTP/1.1
421	52.217349743	:::1	:::1	HTTP	1587	GET /imagen.png HTTP/1.1
508	53.805420457	:::1	:::1	HTTP	1688	GET / HTTP/1.1
514	53.816556481	:::1	:::1	HTTP	1587	GET /imagen.png HTTP/1.1
593	53.869572505	:::1	:::1	HTTP	1688	GET / HTTP/1.1
598	53.878520611	:::1	:::1	HTTP	1587	GET /imagen.png HTTP/1.1
837	54.568691735	:::1	:::1	HTTP	1688	GET / HTTP/1.1
872	63.635995656	:::1	:::1	HTTP	1682	GET / HTTP/1.1
1026	81.259070327	:::1	:::1	HTTP	1688	GET / HTTP/1.1
1121	98.038551357	:::1	:::1	HTTP	1662	GET / HTTP/1.1

En esta imagen se muestra un poco más de información sobre la primera trama, se puede ver que se capturan un total de 1664 bytes, utiliza IPv4 y la IP de origen es la misma que la de destino (127.0.0.1 o localhost) ya que las pruebas las estamos realizando en local, entre otras cosas.

Los puertos que se asignan son, para la IP de origen se asigna el puerto de origen 43500, este puerto es asignado automáticamente por el sistema (puerto dinámico), en cambio, el puerto de destino es el 8080 que es el que le hemos asignado nosotros.

Frame 276: 1664 bytes on wire (13312 bits), 1664 bytes captured (13312 bits) on interface lo, id 0
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
Internet Protocol Version 6, Src: ip6-localhost (::1), Dst: ip6-localhost (::1)
Transmission Control Protocol, Src Port: 43500, Dst Port: 8080, Seq: 1, Ack: 1, Len: 1578
Hypertext Transfer Protocol
GET / HTTP/1.1\r\n
[Expert Info (Chat/Sequence): GET / HTTP/1.1\r\n]
[GET / HTTP/1.1\r\n]
[Severity level: Chat]
[Group: Sequence]
Request Method: GET
Request URI: /
Request Version: HTTP/1.1
Host: localhost:8080\r\n
Connection: keep-alive\r\n
Cache-Control: max-age=0\r\n
sec-ch-ua: "Google Chrome";v="147", "Not.A/Brand";v="8", "Chromium";v="147"\r\n
sec-ch-ua-mobile: ?0\r\n
sec-ch-ua-platform: "Linux"\r\n
Upgrade-Insecure-Requests: 1\r\n
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,/*
Sec-Fetch-Site: none\r\n

Ahora vamos a analizar las tramas del método **POST**. Como se puede observar en la imagen, aparecen un total de 6 tramas; esto se debe a que el servidor se inició 2 veces mientras se realizaba la captura de tráfico. No obstante, el funcionamiento es el esperado: después de 5 consultas introduciendo el correo electrónico, se alcanza el límite y obtenemos el error 403. Estas peticiones tienen un valor de 935 bytes y usan la versión HTTP/1.1. En resumen, el método POST únicamente aparece en el caso en que el usuario introduzca el email.

No.	Time	Source	Destination	Protocol	Length	Info
398	31.363393604	ip6-localhost	ip6-localhost	HTTP	138	POST /accion_form.html HTTP/1.1 (application/x-www-form-urlencoded)
1481	384.531995104	ip6-localhost	ip6-localhost	HTTP	124	POST /accion_form.html HTTP/1.1 (application/x-www-form-urlencoded)
1493	391.816746178	ip6-localhost	ip6-localhost	HTTP	138	POST /accion_form.html HTTP/1.1 (application/x-www-form-urlencoded)
1519	402.833187159	ip6-localhost	ip6-localhost	HTTP	128	POST /accion_form.html HTTP/1.1 (application/x-www-form-urlencoded)
1518	415.359458417	ip6-localhost	ip6-localhost	HTTP	101	POST /accion_form.html HTTP/1.1 (application/x-www-form-urlencoded)
1527	419.808065702	ip6-localhost	ip6-localhost	HTTP	118	POST /accion_form.html HTTP/1.1 (application/x-www-form-urlencoded)

Vamos a ver ahora el error **403** (“Acceso denegado: Demasiadas peticiones”), depende de cuantas veces se reinicie la página hasta que pasen los dos minutos de espera salen más o menos trazas, pero son todas iguales, como se ha visto en la explicación del código el código de error 403 corresponde con “**Forbidden**”, además la traza tiene un tamaño de 598 bytes.

No.	Time	Source	Destination	Protocol	Length	Info
838	54.567259343	ip6-localhost	ip6-localhost	HTTP	598	HTTP/1.1 403 Forbidden (text/html)
873	63.636548589	ip6-localhost	ip6-localhost	HTTP	598	HTTP/1.1 403 Forbidden (text/html)
1031	81.268876074	ip6-localhost	ip6-localhost	HTTP	598	HTTP/1.1 403 Forbidden (text/html)
1123	98.039168883	ip6-localhost	ip6-localhost	HTTP	598	HTTP/1.1 403 Forbidden (text/html)

2. Servicio Apache HTTP con el login y portal de desarrollo para los trabajadores de outlier

1. Instalación de Apache

```
> sudo apt install apache2 -y
```

2. Creación página web con los respectivos ficheros de diseño

```
> sudo nano styles.css
> sudo nano index.html
> sudo nano pagina2.html
```

Fichero HTML `index.html` con el código de estilo de la página principal:

```
<!doctype html>
<html lang="es">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Apache - Servidor Web</title>
    <link rel="preconnect" href="https://fonts.googleapis.com" />
    <link
      href="https://fonts.googleapis.com/css2?family=JetBrains+Mono:wght@400;600&family=Syne:wght@
      rel="stylesheet"
    />
    <link rel="stylesheet" href="styles.css" />
  </head>
  <body>
    <header>
      <div class="logo-row">
        <div class="logo-feather"></div>
        <div class="logo-name">
          Apache HTTP Server
          <br /><span>httpd 2.4 &mdash; puerto 80</span>
        </div>
      </div>
      <span class="badge"> en línea</span>
    </header>

    <main>
      <div class="eyebrow">página principal</div>
      <h1>Servidor <span class="grad">Apache</span><br />funcionando</h1>
      <p class="lead">
        Esta es la página principal servida por Apache HTTP Server. La
        navegación está protegida por autenticación básica con
        <code
          style='
            font-family: "JetBrains Mono", monospace;
            font-size: 0.85em;
            color: #79c0ff;
          '
        >.htaccess</code>
      </p>

      <a href="pagina2.html" class="cta">
        Ir a la página 2
        <span class="arrow">→</span>
      </a>

    </main>
  </body>
</html>
```

```

<div class="cards">
  <div class="card">
    <div class="card-icon"> </div>
    <div class="card-title">Auth Básica</div>
    <div class="card-desc">
      Protección con .htaccess<br />y .htpasswd
    </div>
  </div>
  <div class="card">
    <div class="card-icon"> </div>
    <div class="card-title">Ficheros estáticos</div>
    <div class="card-desc">
      HTML, imágenes y más<br />desde DocumentRoot
    </div>
  </div>
  <div class="card">
    <div class="card-icon"> </div>
    <div class="card-title">MPM Event</div>
    <div class="card-desc">Modelo de proceso<br />asíncrono</div>
  </div>
</div>
</main>

<footer>
  <span>Apache HTTP Server &mdash; UPCT Redes de Datos</span>
  <span>DocumentRoot: /usr/local/apache2/htdocs</span>
</footer>
</body>
</html>

```

Fichero HTML pagina2.html con el código de estilo de la página enlazada:

```

<!doctype html>
<html lang="es">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Página 2 - Apache</title>
    <link rel="preconnect" href="https://fonts.googleapis.com" />
    <link
      href="https://fonts.googleapis.com/css2?family=JetBrains+Mono:wght@400;600"
      "&family=Syne:wght@400;600;700;800&display=swap"
      rel="stylesheet"
    />
    <link rel="stylesheet" href="styles.css" />
  </head>
  <body>
    <header>
      <div class="logo-row">
        <div class="logo-feather"> </div>
        <div class="logo-name">
          Apache HTTP Server
          <br /><span>httpd 2.4 &mdash; puerto 80</span>
        </div>
      </div>
      <nav class="breadcrumb">
        <a href="index.html">inicio</a>
        <span class="sep"></span>
        <span class="current">pagina2.html</span>
      </nav>
    </header>

```

```

<main>
  <div class="page-num">// página 02</div>
  <h1><span class="grad">Segunda</span><br />Página</h1>

  <div class="content-block">
    <p>
      Esta página está enlazada desde
      <code
        style='
          font-family: "JetBrains Mono", monospace;
          color: #79c0ff;
          font-size: 0.85em;
        '
        >index.html</code>
      >
      y es servida directamente por Apache desde el
      <em>DocumentRoot</em>. La navegación entre páginas demuestra el
      servicio de ficheros estáticos y el enlazado entre recursos.
    </p>
    <div class="path-display">GET /pagina2.html HTTP/1.1</div>
  </div>

  <a class="back-btn" href="index.html"> ← Volver al inicio </a>
</main>

<footer>
  <span>Apache HTTP Server &mdash; UPCT Redes de Datos</span>
  <span>DocumentRoot: /usr/local/apache2/htdocs</span>
</footer>
</body>
</html>

```

Fichero CSS styles.css con el código de estilo de diseño de la página:

```

:root {
  --bg: #0d1117;
  --surface: #161b22;
  --border: #30363d;
  --muted: #8b949e;
  --text: #e6edf3;
  --accent: #ff6e40;
  --accent2: #ffa040;
  --mono: "JetBrains Mono", monospace;
  --sans: "Syne", sans-serif;
}

*,
*::before,
*::after {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  background: var(--bg);
  color: var(--text);
  font-family: var(--sans);
  min-height: 100vh;
  display: flex;
  flex-direction: column;

```

```
}

/* Grid de fondo */
body::before {
  content: "";
  position: fixed;
  inset: 0;
  background-image:
    linear-gradient(rgba(255, 110, 64, 0.025) 1px, transparent 1px),
    linear-gradient(90deg, rgba(255, 110, 64, 0.025) 1px, transparent 1px);
  background-size: 40px 40px;
  pointer-events: none;
  z-index: 0;
}

/* Header */
header {
  position: relative;
  z-index: 1;
  padding: 1.5rem 2.5rem;
  border-bottom: 1px solid var(--border);
  display: flex;
  align-items: center;
  justify-content: space-between;
  flex-wrap: wrap;
  gap: 1rem;
}

.logo-row {
  display: flex;
  align-items: center;
  gap: 0.75rem;
}

.logo-feather {
  width: 32px;
  height: 32px;
  background: linear-gradient(135deg, var(--accent), var(--accent2));
  border-radius: 8px;
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 1rem;
}

.logo-name {
  font-weight: 700;
  font-size: 1rem;
  letter-spacing: -0.01em;
}

.logo-name span {
  font-family: var(--mono);
  font-size: 0.75rem;
  color: var(--muted);
  font-weight: 400;
}

.badge {
  font-family: var(--mono);
  font-size: 0.7rem;
  color: var(--accent2);
  border: 1px solid rgba(255, 160, 64, 0.3);
  border-radius: 100px;
  padding: 0.2rem 0.7rem;
}
```

```
    background: rgba(255, 160, 64, 0.05);
}

/* Navigation / Breadcrumb */
.breadcrumb {
  font-family: var(--mono);
  font-size: 0.75rem;
  color: var(--muted);
  display: flex;
  align-items: center;
  gap: 0.5rem;
}
.breadcrumb a {
  color: var(--muted);
  text-decoration: none;
}
.breadcrumb a:hover {
  color: var(--text);
}
.breadcrumb .sep {
  color: #484f58;
}
.breadcrumb .current {
  color: var(--accent);
}

/* Main */
main {
  position: relative;
  z-index: 1;
  max-width: 800px;
  margin: 0 auto;
  padding: 5rem 2rem;
  flex: 1;
}

/* Hero & Typography */
.eyebrow,
.page-num {
  font-family: var(--mono);
  font-size: 0.72rem;
  color: var(--accent);
  text-transform: uppercase;
  letter-spacing: 0.15em;
  margin-bottom: 1rem;
}

.page-num {
  color: var(--muted);
  margin-bottom: 0.75rem;
}

h1 {
  font-size: clamp(2.5rem, 7vw, 5rem);
  font-weight: 800;
  line-height: 1.05;
  letter-spacing: -0.03em;
  margin-bottom: 1.5rem;
}

.grad {
  background: linear-gradient(90deg, var(--accent), var(--accent2));
}
```

```
-webkit-background-clip: text;
-webkit-text-fill-color: transparent;
background-clip: text;
}

.lead {
  font-size: 1.05rem;
  color: var(--muted);
  line-height: 1.8;
  margin-bottom: 3rem;
  max-width: 560px;
}

/* CTA */
.cta {
  display: inline-flex;
  align-items: center;
  gap: 0.6rem;
  background: var(--accent);
  color: #0d1117;
  padding: 0.85rem 2rem;
  border-radius: 10px;
  font-weight: 700;
  font-size: 0.95rem;
  text-decoration: none;
  letter-spacing: -0.01em;
  transition:
    opacity 0.2s,
    transform 0.15s;
  margin-bottom: 4rem;
}

.cta:hover {
  opacity: 0.88;
}

.cta:active {
  transform: scale(0.97);
}

.arrow {
  font-size: 1.1rem;
  transition: transform 0.2s;
}

.cta:hover .arrow {
  transform: translateX(4px);
}

/* Back Button */
.back-btn {
  display: inline-flex;
  align-items: center;
  gap: 0.5rem;
  background: var(--surface);
  border: 1px solid var(--border);
  color: var(--text);
  padding: 0.8rem 1.5rem;
  border-radius: 10px;
  font-weight: 600;
  font-size: 0.9rem;
  text-decoration: none;
  transition:
    border-color 0.2s,
    background 0.2s;
}
```



```
.back-btn:hover {
  border-color: var(--accent);
  background: rgba(255, 110, 64, 0.05);
}

/* Divider */
hr {
  border: none;
  border-top: 1px solid var(--border);
  margin: 0 0 2.5rem;
}

/* Cards */
.cards {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  gap: 1px;
  background: var(--border);
  border: 1px solid var(--border);
  border-radius: 12px;
  overflow: hidden;
}

.card {
  background: var(--surface);
  padding: 1.25rem 1.5rem;
}

.card-icon {
  font-size: 1.4rem;
  margin-bottom: 0.5rem;
}

.card-title {
  font-size: 0.8rem;
  font-weight: 600;
  color: var(--text);
  margin-bottom: 0.25rem;
}

.card-desc {
  font-family: var(--mono);
  font-size: 0.72rem;
  color: var(--muted);
  line-height: 1.5;
}

/* Content Block (Page 2) */
.content-block {
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 16px;
  padding: 2rem;
  margin-bottom: 2rem;
}

.content-block p {
  color: var(--muted);
  line-height: 1.8;
  font-size: 0.95rem;
}
```

```

.path-display {
  font-family: var(--mono);
  font-size: 0.78rem;
  color: #79c0ff;
  background: var(--bg);
  border: 1px solid var(--border);
  border-radius: 8px;
  padding: 0.75rem 1rem;
  margin-top: 1rem;
  display: flex;
  align-items: center;
  gap: 0.5rem;
}
.path-display::before {
  content: "$";
  color: #484f58;
}

/* Footer */
footer {
  position: relative;
  z-index: 1;
  padding: 1.5rem 2.5rem;
  border-top: 1px solid var(--border);
  font-family: var(--mono);
  font-size: 0.72rem;
  color: #484f58;
  display: flex;
  justify-content: space-between;
  flex-wrap: wrap;
  gap: 0.5rem;
}

```

3. Creación de usuarios y establecimiento de contraseñas para autenticación HTTP

```

> sudo htpasswd -c /etc/apache2/passwords fcojaviermercader
New password:
Re-type new password:
Adding password for user fcojaviermercader

> sudo htpasswd -c /etc/apache2/passwords mauromartinezcazaux
New password:
Re-type new password:
Adding password for user mauromartinezcazaux

```

Creación de archivo de contraseñas **passwords**, este archivo con el registro de los usuarios y sus respectivas contraseñas debe ser referenciado en la configuración global del servidor para así activar autenticación.

Contraseñas:

- fcojaviermercader: FJMM2526
- mauromartinezcazaux: MMC2526

Comentar que para la creación de los usuarios y guardado de su contraseña de autenticación HTTP se ha empleado el comando:

```

> sudo htpasswd -c /etc/apache2/passwords <nombre_usuario>
# NOTA: Para posteriores usuarios quitamos el -c

```